SUMMARY OF MODEL TESTS

Test 1: Scratching a plastic pen. The overall accuracy was 90.32%; however, we encountered a problem: we were detecting very few actual errors, capturing only 23% of the anomalies.

> **The solution:** evaluate the data in time windows. If, within a given time period, the error rate is low (for example, 1%), then the model assumes everything is fine and does not trigger an alarm. If, within that same time period, the error rate is high (20%, for example), then an alarm is triggered. This way, even if the model doesn't find all the failures within that window (it only detects 1/4), it's able to realize that something is wrong and alert us. On the other hand, we ensure that a single erroneous data point doesn't cause us to go check the machine when everything was fine.
>
> Furthermore, it was interesting to observe whether this error rate persists across multiple windows before triggering an alarm. This allows us to distinguish between an isolated, unforeseen event and a fault that persists over time and is genuine. To achieve this robustness, the system analyzes the windows every minute, requiring a high percentage of failures across multiple consecutive windows (for example, in 8 of the last 10) to confirm the malfunction.

On the other hand, I found that the material used to rub against the blades had a dramatic effect on the transmission of vibrations. When conducting **Test 2 (Wooden Pencil) and Test 3 (Metal Keychain**), the stiffer impact boosted the accuracy rate: the model detected 100% of the actual anomalies, achieving an overall accuracy of over 98% in both cases.

Finally, in **Test 4 (Live Test)**, I put the entire real-time persistence logic to work. With the script running and analyzing data every 60 seconds, the system reported 0% failures in the first cycles. Upon inserting the pencil, the system quickly recorded 30% failures in just one minute. Upon reaching the risk threshold, the algorithm confirmed a "persistent mechanical failure" and automatically triggered the critical alert to the mobile device via Telegram.

For this live test, I decided to shorten the time window and not require the standard 10 minutes to confirm the failure. The reason is purely practical: maintaining manual friction against the blades for ten minutes straight to trigger the alarm was unfeasible in a demonstration. By adjusting the persistence threshold, I was able to validate that the alert chain works perfectly without having to physically stress the fan for that long.

TEST 1: I'M PUTTING A PLASTIC PEN IN SO IT HITS THE BLADES

Loading files...
Dataset successfully loaded. Total rows: 2458

Total healthy recordings (Baseline): 2402
Total recordings with hidden faults: 56

Training Isolation Forest ONLY with healthy data...

ACCURACY IN DETECTING ANOMALIES: 90.32%

--- ANOMALY REPORT (1 = Healthy, -1 = Fault) ---

	precision	recall	f1-score	support
Fault Detected (-1)	0.59	0.23	0.33	56
Healthy State (1)	0.92	0.98	0.95	481
accuracy			0.90	537
macro avg	0.75	0.61	0.64	537
weighted avg	0.88	0.90	0.88	537

Confusion Matrix:
Row 1: True Defects. Column 1 = Detected Defects, Column 2 = False Negatives (Unseen Defects)
Row 2: True negatives. Column 1 = False alarms, Column 2 = True negatives
[[13 43]
 [9 472]]

13/(13+43)=0.23 VERY LOW PERCENTAGE!!!

Problem: We are detecting very few errors—only 23%.

Solution: Evaluate the data in time windows. If, within a given time period, the failure rate is low (this would need to be defined, but say 1%), then the model assumes everything is fine and does not trigger an alarm. If, during that same time period, the failure rate is high (20%, for example), then an alarm is triggered. This way, even if the model doesn't find all the failures within that window (it only detects 1/4), it's still able to realize that something is wrong and alert us. On the other hand, we ensure that a single erroneous data point doesn't cause us to go check the machine when everything was actually fine.

It would be interesting to see if this error rate persists across several windows before triggering the alarm. This will allow us to distinguish between an isolated, unforeseen event and a fault that persists over time and is genuine. This would need to be studied, but perhaps we could analyze the windows every minute and require that, in the next 10 windows, at least 8 of them show a high failure rate.

TEST 2: I PLACE A WOODEN PENCIL SO IT HITS THE BLADES

Loading files...
Dataset successfully loaded. Total rows: 2449

Total healthy recordings (Baseline): 2402
Total recordings with hidden faults: 47

Training Isolation Forest ONLY with healthy data...

ACCURACY IN DETECTING ANOMALIES: 98.30%

--- ANOMALY REPORT (1 = Healthy, -1 = Fault) ---

	precision	recall	f1-score	support
Fault Detected (-1)	0.84	1.00	0.91	47
Healthy State (1)	1.00	0.98	0.99	481
accuracy			0.98	528
macro avg	0.92	0.99	0.95	528
weighted avg	0.99	0.98	0.98	528

Confusion Matrix:
Row 1: Actual faults. Column 1 = Detected faults, Column 2 = False negatives (unseen faults)
Row 2: Actual healthy. Column 1 = False positives, Column 2 = True negatives
[[47 0]
 [9 472]]

47/(47+0)=1; WE'VE BEEN RIGHT 100% OF THE TIME!!!

It seems that the material has a significant effect (the plastic absorbs some of the vibrations).

PRUEBA 3: METO LLAVERO DE METAL PARA QUE CHOQUE CON LAS ASPAS

Loading files...
Dataset successfully merged. Total rows: 2433

Total healthy recordings (Baseline): 2402
Total recordings with hidden faults: 31

Training Isolation Forest ONLY with healthy data...

ACCURACY IN DETECTING ANOMALIES: 98.24%

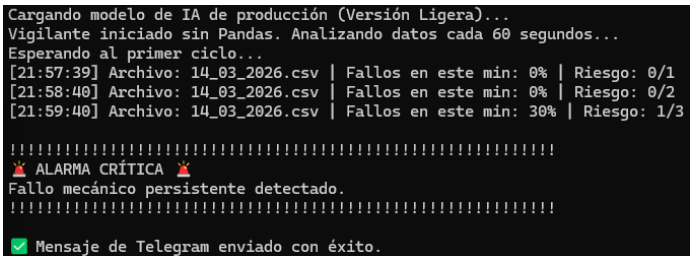
--- ANOMALY REPORT (1 = Healthy, -1 = Fault) ---

	accuracy	recall	f1-score	support
Fault Detected (-1)	0.78	1.00	0.87	31
Healthy State (1)	1.00	0.98	0.99	481
accuracy			0.98	512
macro avg	0.89	0.99	0.93	512
weighted avg	0.99	0.98	0.98	512

Confusion Matrix:
Row 1: Actual failures. Column 1 = Detected failures, Column 2 = False negatives (unseen failures)
Row 2: True negatives. Column 1 = False positives, Column 2 = True negatives
[[31 0]
 [9 472]]

31/(31+0)=1, WE HAVE BEEN CORRECT 100% OF THE TIME!!!

TEST 4: PENCIL TEST – LIVE DEMONSTRATION


Cargando modelo de IA de producción (Versión Ligera)...
Vigilante iniciado sin Pandas. Analizando datos cada 60 segundos...
Esperando al primer ciclo...
[21:57:39] Archivo: 14_03_2026.csv | Fallos en este min: 0% | Riesgo: 0/1
[21:58:40] Archivo: 14_03_2026.csv | Fallos en este min: 0% | Riesgo: 0/2
[21:59:40] Archivo: 14_03_2026.csv | Fallos en este min: 30% | Riesgo: 1/3
!!!

THEORETICAL VALIDATION USING CWRU DATA

```
Extrayendo firmas mecánicas...

Total de grabaciones sanas (Línea Base): 460
Total de grabaciones con averías ocultas: 4140

Entrenando Isolation Forest SOLO con datos sanos...

¡Simulación terminada! PRECISIÓN AL DETECTAR ANOMALÍAS: 91.00%

--- REPORTE DE ANOMALÍAS (1 = Sano, -1 = Avería) ---
                       precision    recall  f1-score   support

Avería Detectada (-1)       1.00      0.91      0.95      4140
      Estado Sano (1)       0.19      1.00      0.33        92

             accuracy                           0.91      4232
            macro avg       0.60      0.95      0.64      4232
         weighted avg       0.98      0.91      0.94      4232


Matriz de Confusión:
Fila 1: Averías reales. Columna 1 = Averías detectadas, Columna 2 = Falsos Sanos (Peligro)
Fila 2: Sanos reales. Columna 1 = Falsas Alarmas, Columna 2 = Sanos correctos
[[3759  381]
 [   0   92]]
```

0% False Alarms: Of the 92 times the evaluated engine was healthy, the AI got it right all 92 times. From a business perspective, this is the most critical metric. A predictive system that generates false alarms ends up being ignored or disabled, as the customer becomes frustrated with sending technicians to inspect machines that are in perfect condition.

The Limits of Physics (False Negatives): The system detected 3,759 failures, but missed 381 (classifying them as healthy). This occurs because the dataset includes minuscule defects of just 0.007 inches. These micro-cracks generate such a slight disturbance that the algorithm still considers them within its “bubble of normality.” In real life, this poses no operational problem: a machine won’t explode in five minutes. As that micro-crack grows over the following weeks, the vibration will exit the bubble, and the AI will detect it well in advance of a collapse.